
● SAVOL 31:

BACKPROPAGATION ALGORITMI QANDAY ISHLAYDI? GRADIENT DESCENT , O'QISH TEZLIGI (LEARNING RATE) VA UNING MODELGA TA'SIRI. LEARNING RATE JUDA KATTA YOKI KICHIK BO'LSA NIMA SODIR BO'LADI?

1. O'rganish Muammosi — Nima Topish Kerak?

Neyron tarmoq o'rganishi = **Vaznlarni (w) va biaslarni (b) to'g'ri topish.**

Maqsad:

Model bashorati ($\hat{y}$) $\rightarrow$ Haqiqiy qiymat (y) ga yaqin bo'lsin

Savol:

Millionlab vaznlarni qanday to'g'irlash kerak?

Qaysi tomonga o'zgartirish kerak?

Qanchalik o'zgartirish kerak?

Javob:

Gradient Descent + Backpropagation!

2. Xato Funksiyasi (Loss Function) — O'rganishning Kompası

Xato funksiyasi – model qanchalik yomon ishlayotganini o'lchaydi.

Regressiya uchun MSE:

$$L = (1/n) \sum (y_i - \hat{y}_i)^2$$

Binary klassifikatsiya:

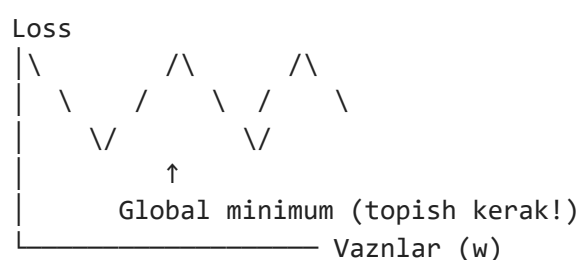
$$L = -(y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y}))$$

Ko'p sinf klassifikatsiya:

$$L = -\sum y_j \cdot \log(\hat{y}_j)$$

Maqsad: L ni MINIMALLASH

Loss funksiyasi landshafti:



Muammo: Millionlab o'lchovli fazoda minimum topish

- Analitik yechim mumkin emas
- Iterativ yondashuv kerak → Gradient Descent

3. Gradient Descent — Asosiy Algoritm

Gradient — funksiyaning eng tez o'sish yo'nalishi. Manfiy gradient — eng tez kamayish yo'nalishi.

Analogiya:

Tog'da ko'zi ko'r odam eng pastga tushmoqchi:

- Har qadamda oyoq ostini his qiladi (gradient)
- Eng qiyalik tomonga bir qadam bosadi
- Asta-sekin vodiya tushadi

Gradient Descent:

- Loss landshaftida "pastga tushish"
- Gradient → Qaysi tomonga o'zgartirish kerakligini aytadi
- Learning rate → Qadam qanchalik katta bo'lishini belgilaydi

Matematik formula:

Vaznlarni yangilash:

$$w_{\text{yangi}} = w_{\text{eski}} - \alpha \cdot \partial L / \partial w$$

α → Learning rate (o'qish tezligi)

$\partial L / \partial w$ → Loss ning w ga nisbatan gradienti
("w ozgina oshsa, L qanchalik o'zgaradi?")

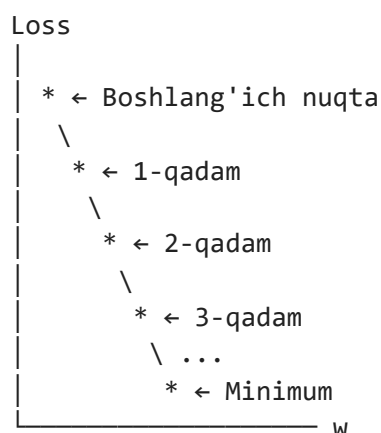
Manfiy belgi (-):

Gradient → O'sish yo'nalishi

Biz → Kamayish istalaymiz

Shuning uchun gradientga TESKARI tomonga boramiz

Grafik:



$$\text{Har qadam: } w = w - \alpha \cdot \partial L / \partial w$$

4. Gradient Descent Turlari

Batch Gradient Descent:

- Barcha ma'lumot bir vaqtda
- Aniq gradient
- Juda sekin (katta ma'lumotda)
- Xotira ko'p

Stochastic GD (SGD):

- Har safar 1 ta namuna
- Tez, lekin "shovqinli" (beqaror)
- Lokal minimumdan chiqib ketishi mumkin (afzallik!)
- Katta ma'lumotda foydali

Mini-batch GD (Standart):

- Har safar k ta namuna (batch_size=32, 64, 128)
- Ikkalasining yaxshi tomoni
- GPU parallel hisoblash uchun ideal
- Zamonaviy tarmoqlarda STANDART

Taqqoslash:

Batch: —————→ (to'g'ri, lekin sekin)

SGD: ~~~~~→ (tez, lekin "shovqinli")

Mini-batch: ~~~~~→ (muvozanatli) ✓

5. Backpropagation — Gradientni Hisoblash

Backpropagation = Zanjir qoidasi (Chain Rule) ni tarmoqqa qo'llash.

Muammo:

Tarmoq:

$x \rightarrow [W_1] \rightarrow h_1 \rightarrow [W_2] \rightarrow h_2 \rightarrow [W_3] \rightarrow \hat{y} \rightarrow L$

Savol: $\partial L / \partial W_1 = ?$

(Birinchi qatlamdagi vaznlar Loss ga qanday ta'sir qiladi?)

To'g'ridan-to'g'ri hisoblash qiyin:

$W_1 \rightarrow h_1 \rightarrow h_2 \rightarrow \hat{y} \rightarrow L$

(Ko'p qatlam orqali o'tish kerak)

Zanjir qoidasi (Chain Rule):

$$\partial L / \partial W_1 = \partial L / \partial \hat{y} \cdot \partial \hat{y} / \partial h_2 \cdot \partial h_2 / \partial h_1 \cdot \partial h_1 / \partial W_1$$

Har bo'g'in alohida hisoblanadi!

Backpropagation jarayoni:

OLDINGA O'TISH (Forward Pass):

$x \rightarrow W_1 \rightarrow h_1 \rightarrow W_2 \rightarrow h_2 \rightarrow W_3 \rightarrow \hat{y} \rightarrow L$

(Bashorat qilinadi, Loss hisoblanadi)

Har qatlamdagi oraliq qiymatlar SAQLANADI

ORQAGA O'TISH (Backward Pass):

$L \leftarrow W_3 \leftarrow h_2 \leftarrow W_2 \leftarrow h_1 \leftarrow W_1 \leftarrow x$
(Gradientlar orqaga tarqatiladi)

1. $\partial L / \partial W_3$ — Eng oxirgi qatlam gradienti (oson!)
2. $\partial L / \partial W_2$ — Chain rule bilan
3. $\partial L / \partial W_1$ — Chain rule bilan (eng qiyin)

Natija: Barcha vaznlar uchun gradient topiladi

Sodda misol — 2 qatlamli tarmoq:

Forward:

$z_1 = W_1 \cdot x + b_1$
 $h_1 = \text{ReLU}(z_1)$
 $z_2 = W_2 \cdot h_1 + b_2$
 $\hat{y} = \text{Sigmoid}(z_2)$
 $L = -(y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y}))$

Backward:

$\partial L / \partial \hat{y} = -(y / \hat{y} - (1-y) / (1-\hat{y}))$
 $\partial L / \partial z_2 = \partial L / \partial \hat{y} \cdot \hat{y} \cdot (1-\hat{y}) \quad \leftarrow \text{Sigmoid}'$
 $\partial L / \partial W_2 = \partial L / \partial z_2 \cdot h_1^T$
 $\partial L / \partial h_1 = W_2^T \cdot \partial L / \partial z_2$
 $\partial L / \partial z_1 = \partial L / \partial h_1 \cdot \text{ReLU}'(z_1) \quad \leftarrow \text{ReLU}'$
 $\partial L / \partial W_1 = \partial L / \partial z_1 \cdot x^T$

Yangilash:

$W_1 = W_1 - \alpha \cdot \partial L / \partial W_1$
 $W_2 = W_2 - \alpha \cdot \partial L / \partial W_2$

6. Vanishing va Exploding Gradient

Vanishing Gradient (Gradient yo'qolishi):

Zanjir qoidasida ko'paytirish:
 $\partial L / \partial W_1 = \delta_3 \cdot \delta_2 \cdot \delta_1 \cdot x$

$\text{Sigmoid}' \leq 0.25 \rightarrow 0.25 \times 0.25 \times 0.25 = 0.0156$
100 qatlam: $0.25^{100} \approx 6 \times 10^{-61} \rightarrow$ Gradient yo'qoldi!

Natija: Birinchi qatlamlar O'RGANMAYDI

Yechim: ReLU aktivatsiya, Batch Normalization, Skip connection

Exploding Gradient (Gradient portlashi):

Vaznlar katta bo'lsa: $2 \times 2 \times 2 \times \dots = 2^{100} \rightarrow \infty$
Natija: Vaznlar INF yoki NaN bo'lib qoladi
Yechim: Gradient Clipping (gradient max qiymatga cheklanadi)

7. Learning Rate — Eng Muhim Giperparametr

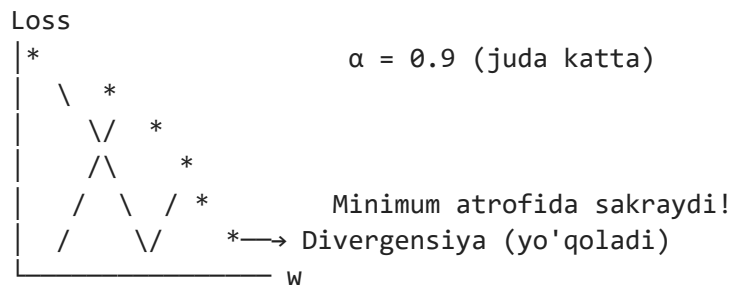
Learning rate (α) — har yangilashda qadam qanchalik katta bo'lishini belgilaydi.

$$w = w - \alpha \cdot \partial L / \partial w$$

↑

Shu α qanchalik katta/kichik?

Juda Katta Learning Rate:



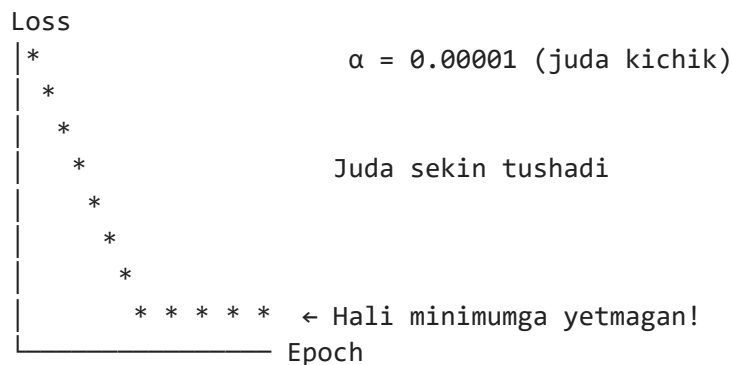
Nima bo'ladi:

- Minimumni "otib o'tadi"
- Loss kamayish o'rniga OSHADI
- Tarmoq o'rganmaydi, divergensiya ketadi
- Vaznlar NaN bo'lib qolishi mumkin

Belgilari:

- Loss grafikda keskin oshadi
- Training to'xtaydi
- "Loss is NaN" xabari

Juda Kichik Learning Rate:



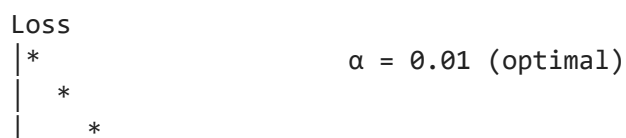
Nima bo'ladi:

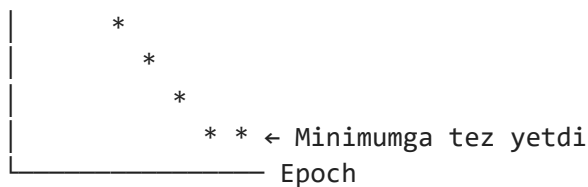
- Juda sekin o'rganadi
- Ko'p vaqt va resurs ketadi
- Vaqt chegarasida yaxshi natijaga erisha olmaydi
- Lokal minimumda "qolib ketishi" mumkin

Belgilari:

- Loss juda sekin kamayadi
- 1000 epoch o'tsa ham yaxshi natija yo'q

Optimal Learning Rate:





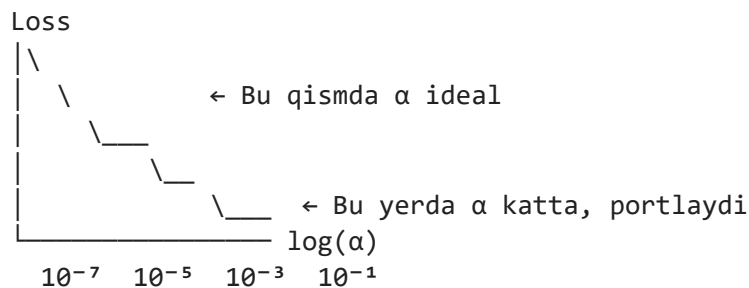
Belgilari:

- Loss doimiy, silliq kamayadi
- Ma'lum epochdan keyin to'xtaydi (plateau)
- Train va Validation loss birga tushadi

8. Learning Rate Tanlash Usullari

Learning Rate Finder:

G'oya: α ni juda kichikdan boshlab, asta-asta oshirib borish
 Loss eng tez kamaygan $\alpha \rightarrow$ Optimal!



Optimal $\alpha \rightarrow$ Loss eng qiyalik nuqtasi (odatda 10^{-3} yoki 10^{-4})

Learning Rate Schedulers:

Step Decay:

- Har N epochda α ni kamaytirish
- $\alpha = \alpha_0 \times 0.1^{(\text{epoch}/30)}$
- Boshida tez, keyin silliq sozlanadi

Exponential Decay:

$$\alpha = \alpha_0 \times e^{(-k \cdot \text{epoch})}$$

Cosine Annealing:

- $\alpha = \alpha_{\min} + 0.5 \cdot (\alpha_{\max} - \alpha_{\min}) \cdot (1 + \cos(\pi \cdot t/T))$
- Cosinus shakli: yuqori → past → yuqori (tsiklik)
- Lokal minimumdan chiqishga yordam beradi

Warm-up + Decay:

- Avval α ni oshir (warm-up) → Keyin kamayt (decay)
- Transformer modellarda STANDART (GPT, BERT)

Adaptive Methods (Optimizer ichida):

- Adam, RMSprop → Har vazn uchun α avtomatik sozlanadi
- Qo'lda sozlash kamroq kerak

9. Zamonaviy Optimizerlar

Oddiy SGD dan yaxshiroq optimizerlar:

Momentum:

$$v = \beta \cdot v - \alpha \cdot \partial L / \partial w \quad \leftarrow \text{Tezlik yig'iladi}$$

$$w = w + v$$

$$\beta = 0.9 \text{ (standart)}$$

Fizik analogiya: Shar tog'dan tushayotganda tezlik yig'adi

→ Kichik to'siqlarni "engib o'tadi"

→ Gradient shovqiniga chidamli

RMSprop:

Har vazn uchun ALOHIDA α

Ko'p o'zgarmagan vaznlar → Katta α

Ko'p o'zgangan vaznlar → Kichik α

→ Masofa topografiyasiga moslashadi

Adam (Adaptive Moment Estimation):

Momentum + RMSprop = Adam

$$m = \beta_1 \cdot m + (1 - \beta_1) \cdot g \quad \leftarrow \text{1-moment (Momentum)}$$

$$v = \beta_2 \cdot v + (1 - \beta_2) \cdot g^2 \quad \leftarrow \text{2-moment (RMSprop)}$$

$$w = w - \alpha \cdot \hat{m} / \sqrt{\hat{v} + \epsilon}$$

$\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=10^{-8}$ → Standart qiymatlar

→ ZAMONAVIY STANDART optimizer

→ Ko'p holatlarda $\alpha=0.001$ bilan zo'r ishlaydi

10. To'liq O'rganish Jarayoni

```
import torch
import torch.nn as nn

# Model
model = nn.Sequential(
    nn.Linear(784, 256),
    nn.ReLU(),
    nn.Linear(256, 128),
    nn.ReLU(),
    nn.Linear(128, 10),
    nn.Softmax(dim=1)
)

# Optimizer va Loss
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
criterion = nn.CrossEntropyLoss()

# O'rgatish jarayoni:
for epoch in range(100):
    for X_batch, y_batch in dataloader:

        # 1. Forward Pass
        y_pred = model(X_batch)
```

```

# 2. Loss hisoblash
loss = criterion(y_pred, y_batch)

# 3. Gradientlarni nolga qaytarish (muhim!)
optimizer.zero_grad()

# 4. Backpropagation
loss.backward()    # ← Barcha  $\partial L / \partial w$  hisoblanadi

# 5. Vaznlarni yangilash
optimizer.step()   # ←  $w = w - \alpha \cdot \partial L / \partial w$ 

```

```
print(f"Epoch {epoch}: Loss={loss.item():.4f}")
```

Nima uchun zero_grad() kerak:

PyTorch gradientlarni YIG'IB BORADI (akkumulatsiya)

zero_grad() chaqirilmasa:

1-batch gradient: $\partial L / \partial w = 0.5$

2-batch gradient: $\partial L / \partial w = 0.5 + 0.3 = 0.8$ ← Noto'g'ri!

3-batch gradient: $\partial L / \partial w = 0.8 + 0.4 = 1.2$ ← Yanada noto'g'ri!

zero_grad() bilan:

1-batch: $\partial L / \partial w = 0.5 \rightarrow$ Yangilash $\rightarrow 0$ ga qaytarish

2-batch: $\partial L / \partial w = 0.3 \rightarrow$ Yangilash $\rightarrow 0$ ga qaytarish 

11. Xulosa

Gradient Descent:

$w = w - \alpha \cdot \partial L / \partial w$

Loss landshaftida pastga tushish

Mini-batch GD $\rightarrow$ Zamonaviy standart

Backpropagation:

Forward Pass $\rightarrow$ Bashorat + Loss hisoblash

Backward Pass $\rightarrow$ Zanjir qoidasi bilan gradientlar

Barcha qatlamlarda $\partial L / \partial w$ topiladi

$\rightarrow$ Vaznlar yangilanadi

Learning Rate (α):

Juda katta $\rightarrow$ Divergensiya, NaN, oscillatsiya

Juda kichik $\rightarrow$ Sekin, lokal minimumda qolish

Optimal $\rightarrow$ Silliq kamayish, tez konvergensiya

LR tanlash:

LR Finder $\rightarrow$ Eng yaxshi α ni topish

Schedulers $\rightarrow$ Vaqt o'tishi bilan kamaytirish

Adam $\rightarrow$ Avtomatik moslashuvchi $\rightarrow$ Standart

Vanishing/Exploding:

Vanishing $\rightarrow$ ReLU, BatchNorm, Skip connection

Exploding $\rightarrow$ Gradient Clipping

💡 **Eslab qol:** Backpropagation — "aybdorni topish" algoritmi. Loss katta bo'ldi → Kimning "aybi"? Zanjir qoidasi orqali har vaznning hisssasi aniqlanadi → Aybdorlar tuzatiladi. Bu jarayon millionlab parametr uchun bir vaqtda, har epochda takrorlanadi — bu zamonaviy AI ning matematikasi!

● SAVOL 32:

SCIKIT-LEARN KUTUBXONASINING ASOSIY TUZILISHI: ESTIMATORS , TRANSFORMERS , PIPELINES . FIT() , PREDICT() , SCORE() METODLARI VA ULARNING BARCHA ALGORITMLARDA BIR XIL INTERFEYS BERISHI.

1. Scikit-learn — Falsafa va Dizayn

Scikit-learn ning eng kuchli tomoni — **barcha algoritmlar bir xil interfeys** orqali ishlaydi. Bu "Consistent API" deyiladi.

Boshqa kutubxonalarda:

```
LinearRegression → .train(), .inference()
KMeans           → .cluster(), .get_labels()
SVM              → .fit_svm(), .classify()
→ Har algoritm o'z usulida → Chalkash!
```

Scikit-learn da:

```
LinearRegression → .fit(), .predict()
KMeans           → .fit(), .predict()
SVM              → .fit(), .predict()
RandomForest     → .fit(), .predict()
→ BARCHA bir xil! → Kod o'zgartirilmaydi
```

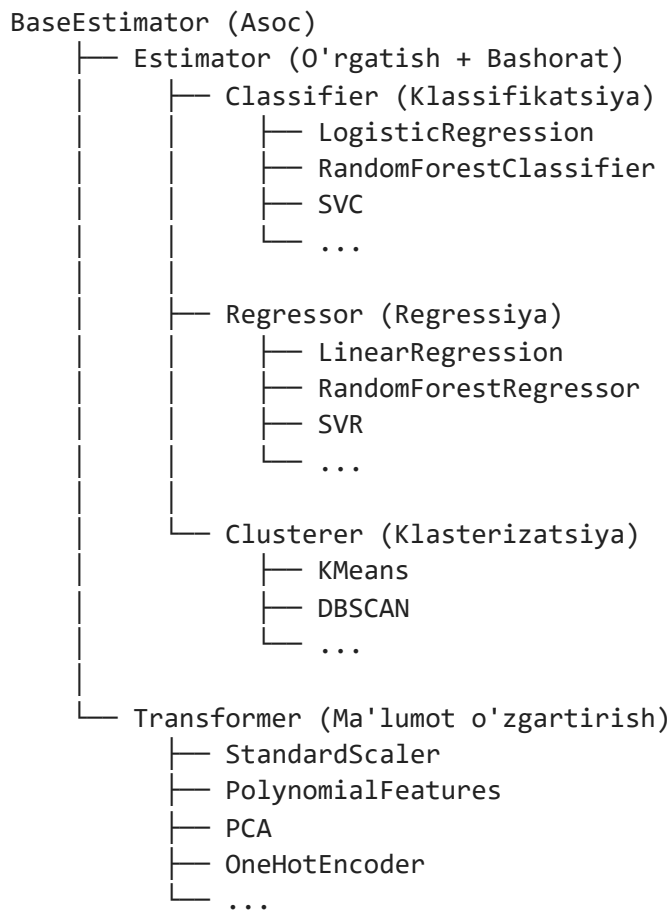
Bu dizaynning afzalligi:

1. O'rganish oson:
Bir interfeys o'rgan → Barcha algoritmlarni bilasan
2. Algoritm almashtirish oson:

```
model = LinearRegression() # Bu qatorni
model = RandomForest()     # Shu bilan almashtirsang — ishлайди!
```
3. Pipeline qurilishi oson:
Har qadam bir xil interfeys → Zanjirga ulash mumkin
4. Giperparametr qidirish oson:
GridSearchCV → Har model uchun bir xil ishlaydi

2. Scikit-learn Iyerarxiyasi

Scikit-learn ob'ektlari:



3. Estimator — O'rgatish va Bashorat

Estimator — `fit()` metodi mavjud bo'lgan har qanday ob'ekt.

`fit()` — O'rgatish:

```
model.fit(X_train, y_train)
```

Nazariy asos:

`fit()` ichida nima bo'ladi:

`LinearRegression.fit()`:

- $\beta = (X^T X)^{-1} \cdot X^T y$ (OLS yechimi)
- `model.coef_`, `model.intercept_` saqlanadi

`RandomForest.fit()`:

- 100 ta Decision Tree quriladi
- Har daraxt bootstrap namunada o'rganadi
- `model.estimators_` saqlanadi

`KMeans.fit()`:

- Sentroidlar iterativ topiladi
- `model.cluster_centers_` saqlanadi

`StandardScaler.fit()`:

- μ va σ hisoblanadi (X o'zgartirilmaydi!)
- `scaler.mean_`, `scaler.scale_` saqlanadi

Muhim: `fit()` → Faqat TRAIN ma'lumotida chaqiriladi!
Test ma'lumotida chaqirish → Data Leakage!

predict() — Bashorat:

```
y_pred = model.predict(X_test)
```

`predict()` ichida nima bo'ladi:

`LinearRegression.predict()`:
→ $\hat{y} = X \cdot \hat{\beta}$ (`fit()` da topilgan β ishlatiladi)

`RandomForest.predict()`:
→ Har daraxt ovoz beradi
→ Ko'pchilik ovozi → Yakuniy sinf

`KMeans.predict()`:
→ Har nuqtani eng yaqin sentroidga tayinlaydi

Muhim: `predict()` `fit()` DAN KEYIN chaqiriladi!
`fit()` siz `predict()` → `NotFittedError`

predict_proba() — Ehtimolliklar:

```
# Faqat Klassifikatorlarda
ehtimolliklar = model.predict_proba(X_test)
# [[0.9, 0.1], ← 1-namuna: 90% sinf 0
#  [0.2, 0.8], ← 2-namuna: 80% sinf 1
#  ...]
```

```
# Bir sinfning ehtimolligi:
sinf_1_ehtimol = ehtimolliklar[:, 1]
```

score() — Baholash:

```
skor = model.score(X_test, y_test)
```

`score()` – Har model uchun o'z metrikasi:

`Classifier.score()` → Accuracy (to'g'rilik)
`Regressor.score()` → R^2 koeffitsienti
`Clusterer.score()` → Inertia (manfiy)

Muhim: `score()` faqat umumiy baholash uchun.
Batafsil baholash → `metrics` modulidan:
`precision_score()`, `recall_score()`, `f1_score()`...

4. Transformer — Ma'lumot O'zgartirish

Transformer — `fit()` + `transform()` metodlari mavjud ob'ekt.

```
# Transformer ish jarayoni:
```

```
# 1. O'rganish (statistika hisoblash):
scaler.fit(X_train)
# →  $\mu = [23.5, 150.2, 0.7]$ 
# →  $\sigma = [5.2, 30.1, 0.1]$ 

# 2. O'zgartirish:
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
# →  $(X - \mu) / \sigma \leftarrow$  O'rganilgan  $\mu$  va  $\sigma$  ishlatiladi

# 3. Qisqartirish (fit + transform):
X_train_scaled = scaler.fit_transform(X_train)
# Ikki qadam bitta!
```

Juda Muhim Qoida:

✗ NOTO'G'RI:

```
scaler.fit(X_train)
scaler.fit(X_test) ← Test da fit() → Data Leakage!
```

✓ TO'G'RI:

```
scaler.fit(X_train) ← Faqat train da
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test) ← Bir xil scaler
```

Nima uchun?

Test ma'lumoti "real hayot" ni ifodalaydi.
 Real hayotda yangi ma'lumot uchun
 train statistikasi ishlatiladi.
 Test da fit() qilsak → Test ma'lumoti "ko'rilgan" bo'ladi.

Asosiy Transformerlar:

Miqyoslash:

```
StandardScaler →  $(X - \mu) / \sigma \rightarrow N(0,1)$ 
MinMaxScaler →  $(X - \min) / (\max - \min) \rightarrow [0,1]$ 
RobustScaler → Median va IQR → Outlierlarga bardoshli
```

Kodlash:

```
OneHotEncoder → Kategoriyali → Binary vektorlar
LabelEncoder → Kategoriyali → Butun sonlar
OrdinalEncoder → Tartibli kategoriya → Sonlar
```

O'lcham kamaytirish:

```
PCA → Asosiy komponentalar
TruncatedSVD → Siyrak matritsalar uchun
```

Xususiyat yaratish:

```
PolynomialFeatures →  $X, X^2, X \cdot Y, \dots$ 
KBinsDiscretizer → Uzluksiz → Diskret
```

Matn:

```
TfidfVectorizer → Matn → Raqamli vektor
CountVectorizer → So'z chastotasi
```

5. Pipeline — Barcha Qadamlarni Birlashtirish

Pipeline — bir necha transformer va estimatorni zanjirga ulash. Bu scikit-learn ning **eng kuchli** xususiyati.

Pipeline siz muammo:

❌ *Pipeline siz – Xavfli va noqulay:*

Train:

```
scaler.fit(X_train)
X_train_s = scaler.transform(X_train)
```

```
poly.fit(X_train_s)
X_train_p = poly.transform(X_train_s)
```

```
model.fit(X_train_p, y_train)
```

Test:

```
X_test_s = scaler.transform(X_test)
X_test_p = poly.transform(X_test_s)
y_pred = model.predict(X_test_p)
```

Muammo: 6 ta qadam → Biron qadamni unutsang → Xato!

CrossValidation da har fold uchun qo'lda takrorlash → Data Leakage!

Pipeline bilan — To'g'ri:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.linear_model import LinearRegression
```

Pipeline yaratish:

```
pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("poly", PolynomialFeatures(degree=2)),
    ("model", LinearRegression())
])
```

Nomlar ixtiyoriy, lekin ma'noli bo'lsin

O'rgatish – Barcha qadamlar avtomatik:

```
pipe.fit(X_train, y_train)
# 1. scaler.fit_transform(X_train)
# 2. poly.fit_transform(scaled_X)
# 3. model.fit(poly_X, y_train)
```

Bashorat – Barcha qadamlar avtomatik:

```
y_pred = pipe.predict(X_test)
# 1. scaler.transform(X_test) ← fit() emas!
# 2. poly.transform(scaled_X)
# 3. model.predict(poly_X)
```

Baholash:

```
skor = pipe.score(X_test, y_test)
```

Pipeline ning Data Leakage himoyasi:

Cross-Validation + Pipeline:

```
fold 1: Train=[2,3,4,5], Test=[1]
→ pipe.fit(X[2,3,4,5]) ← Faqat bu fold train da scaler o'rganadi
→ pipe.predict(X[1]) ← O'sha scaler bilan test transform qilinadi
```

```
fold 2: Train=[1,3,4,5], Test=[2]
→ pipe.fit(X[1,3,4,5]) ← Yangi fold, yangi scaler
→ pipe.predict(X[2])
```

Har fold uchun scaler qaytadan o'rganadi → Data Leakage YO'Q ✅

Pipeline siz:

```
scaler.fit(X_train) ← Butun train da fit
cross_val_score(model, X_scaled, y)
→ Barcha foldlarda bir xil scaler → Test ma'lumoti "ko'rilgan"!
```

6. ColumnTransformer — Turli Ustunlar Uchun

Haqiqiy ma'lumotda bir xil ishlov berish kamdan-kam. Har ustun turiga mos transformer kerak:

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline

# Ustunlar bo'yicha ajratish:
raqamli_ustunlar = ["yosh", "daromad", "tajriba"]
kategoriyali_ustunlar = ["shahar", "kasb", "jinsi"]

# Raqamli ustunlar uchun:
raqamli_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")), # Bo'sh qiymat
    ("scaler", StandardScaler())
])

# Kategoriyali ustunlar uchun:
kategoriyali_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

# ColumnTransformer – Birlashtirish:
preprocessor = ColumnTransformer([
    ("raqamli", raqamli_pipe, raqamli_ustunlar),
    ("kategoriyali", kategoriyali_pipe, kategoriyali_ustunlar)
])

# To'liq Pipeline:
to_liq_pipe = Pipeline([
    ("preprocessor", preprocessor),
    ("model", RandomForestClassifier(n_estimators=100))
])
```

```
to_liq_pipe.fit(X_train, y_train)
y_pred = to_liq_pipe.predict(X_test)
```

7. GridSearchCV — Giperparametr Qidirish

Pipeline + GridSearchCV = Avtomatik eng yaxshi model topish:

```
from sklearn.model_selection import GridSearchCV

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", RandomForestClassifier())
])

# Giperparametrlar – "qatlam__parametr" formati:
params = {
    "model__n_estimators": [50, 100, 200],
    "model__max_depth":    [3, 5, 10, None],
    "model__min_samples_leaf": [1, 2, 5],
}

grid = GridSearchCV(
    pipe,
    params,
    cv=5,                # 5-fold cross-validation
    scoring="f1",        # F1 bo'yicha optimallashtirish
    n_jobs=-1,          # Parallel hisoblash
    verbose=1
)

grid.fit(X_train, y_train)

print(f"Eng yaxshi parametrlar: {grid.best_params_}")
print(f"Eng yaxshi CV skori:    {grid.best_score_:.4f}")

# Eng yaxshi model:
eng_yaxshi = grid.best_estimator_
y_pred = eng_yaxshi.predict(X_test)
```

8. set_params() va get_params() — Moslashtiruvchanlik

```
# Parametrlarni ko'rish:
pipe.get_params()
# {
#   'scaler': StandardScaler(),
#   'scaler__copy': True,
#   'scaler__with_mean': True,
#   'model': RandomForestClassifier(),
#   'model__n_estimators': 100,
#   ...
# }

# Parametrlarni o'zgartirish:
```

```
pipe.set_params(model__n_estimators=200)
pipe.set_params(model=GradientBoostingClassifier())
```

Bu GridSearchCV va clone() uchun muhim!

9. Custom Transformer — O'z Transformerimiz

Scikit-learn interfeysi kengaytiriladigan:

```
from sklearn.base import BaseEstimator, TransformerMixin
import numpy as np

class LogTransformer(BaseEstimator, TransformerMixin):
    """Musbat qiymatlarni log ga aylantiradi"""

    def __init__(self, epsilon=1e-8):
        self.epsilon = epsilon # Noldan himoya

    def fit(self, X, y=None):
        # Log transformer uchun fit da hech narsa kerak emas
        return self # ← Har doim self qaytarish!

    def transform(self, X):
        return np.log(X + self.epsilon)

    def inverse_transform(self, X):
        return np.exp(X) - self.epsilon

# Ishlatish:
pipe = Pipeline([
    ("log", LogTransformer()),
    ("scaler", StandardScaler()),
    ("model", LinearRegression())
])
pipe.fit(X_train, y_train) # ✅ To'liq ishlaydi!
```

10. Xulosa

Scikit-learn tuzilishi:

Estimator:

- fit(X, y) → Modelni o'rgatish (parametrlar topiladi)
- predict(X) → Bashorat qilish
- score(X, y) → Baholash (Accuracy yoki R^2)

Transformer:

- fit(X) → Statistika o'rganish (μ , σ , min, max...)
- transform(X) → Ma'lumotni o'zgartirish
- fit_transform → Ikkalasini birga

Pipeline:

- [("nom1", transformer1),
- ("nom2", transformer2),


```
("nom3", estimator)]
```

```
fit()      → Barcha qadamlar ketma-ket fit+transform  
predict() → Barcha qadamlar transform, oxirgi predict  
→ Data Leakage himoyasi avtomatik!
```

ColumnTransformer:

Har ustun turiga mos ishlov berish

Raqamli → StandardScaler

Kategoriyali → OneHotEncoder

Asosiy qoidalar:

- ✓ fit() → FAQAT train ma'lumotida
- ✓ Pipeline → Har doim ishlatish (Data Leakage uchun)
- ✓ GridSearchCV + Pipeline → Giperparametr qidirish
- ✓ Consistent API → Algoritm almashtirishda faqat 1 qator

💡 **Eslab qol:** Scikit-learn ning "Consistent API" — bu nafaqat qulaylik, balki **xavfsizlik** ham. Pipeline ishlatish Data Leakage ni avtomatik oldini oladi. "Hamma narsani Pipeline orqali qil" — bu mashinali o'rganishdagi eng muhim amaliy qoida!

● SAVOL 33:

CROSS-VALIDATION (K-FOLD) NIMA VA NIMA UCHUN BIR MARTALIK TRAIN-TEST BO'LINISHIDAN YAXSHIROQ? K=5 VA K=10 NING FARQLARI, KICHIK DATASETLARDA QO'LLASH.

1. Bir Martalik Train-Test ning Muammosi

Oddiy Train-Test bo'linish:

Ma'lumot: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

↓

Train [1..8] + Test [9, 10]

Muammo 1 — "Omad" yoki "Omadsizlik":

Test [9, 10] → Oson namunalar → $R^2=0.95$ (yaxshi ko'rinadi)

Test [9, 10] → Qiyin namunalar → $R^2=0.45$ (yomon ko'rinadi)

Haqiqiy model sifati? → Bilmaymiz!

Natija bitta tasodifiy bo'linishga bog'liq.

Muammo 2 — Ma'lumot isrofgarligi:

$n=100$ ma'lumot → 80 train, 20 test

Test uchun 20 ta qimmatli namuna "sarf" bo'ladi

Kichik datasetda bu juda muhim!

Muammo 3 — Ishonchlilik yo'q:

Bitta skor → Qanchalik beqaror ekanini bilmaymiz

Skor = 0.85, lekin ± 0.15 bo'lishi mumkin?

2. K-Fold Cross-Validation — G'oya

K-Fold — ma'lumotni K teng bo'lakka ajratib, har safar boshqa bo'lak test sifatida ishlatish.

k=5 misol:

Ma'lumot: [] (100 ta)
↓ 5 teng bo'lakka ajratish ↓

Bo'lak 1: []

Bo'lak 2: []

Bo'lak 3: []

Bo'lak 4: []

Bo'lak 5: []

Iteratsiya 1: Train[2,3,4,5] Test[1] → Score₁=0.88

Iteratsiya 2: Train[1,3,4,5] Test[2] → Score₂=0.91

Iteratsiya 3: Train[1,2,4,5] Test[3] → Score₃=0.85

Iteratsiya 4: Train[1,2,3,5] Test[4] → Score₄=0.89

Iteratsiya 5: Train[1,2,3,4] Test[5] → Score₅=0.92

Final: O'rtacha = 0.89 ± 0.03 (standart og'ish)

Muhim natija:

0.89 → Model sifati (ishonchli!)

±0.03 → Beqarorlik darajasi (kichik → yaxshi!)

Nima uchun yaxshiroq:

1. Har namuna bir marta TEST, k-1 marta TRAIN da ishlatiladi
→ Ma'lumot isrof bo'lmaydi
2. k ta skor → O'rtacha va standart og'ish
→ "Model 0.89±0.03 ishlaydi" → Ishonchli bayonot
3. Tasodifiy bo'linishga bog'liqlik yo'q
→ Natija barqaror va takrorlanuvchi
4. Butun dataset baholanadi
→ "Omadli test" muammosi yo'q

3. Bias-Variance CV kontekstida

k qiymati → Bias va Variance o'rtasidagi tradeoff:

k kichik (k=2, k=3):

Train kichik → Model kamroq o'rganadi

→ Baholash PESSIMISTIK (haqiqiydan past)

→ Yuqori Bias (baholashda)

k katta (k=n, ya'ni LOOCV):

Train deyarli to'liq → Model yaxshi o'rganadi

→ Baholash OPTIMISTIK ga yaqin

- Past Bias, lekin Yuqori Variance
- Har k da model boshqacha → Natijalar beqaror

k=5 yoki k=10:

- Empirik tadqiqotlar ko'rsatgan "oltin o'rta"
- Bias ham, Variance ham maqbul darajada

4. k=5 va k=10 — Taqqoslash

	k=5	k=10
Train ulushi:	80%	90%
Test ulushi:	20%	10%
Iteratsiyalar:	5	10
Hisoblash vaqti:	1x	~2x
Bias (baholash):	Biroz yuqori	Pastroq
Variance:	Pastroq	Biroz yuqori
Natija barqarorligi:	Yaxshi	Yaxshi
Kichik dataset:	Afzalroq	Ham yaxshi
Katta dataset:	Standart	Sekinroq

Qachon k=5:

- Katta dataset ($n > 10,000$)
- Hisoblash resursi cheklangan
- Tez prototiplash
- sklearn standart (cv=5)
- Vaqt muhim bo'lganda

Qachon k=10:

- O'rta dataset ($1,000 < n < 10,000$)
- Aniqroq baholash kerak
- Final model taqqoslashida
- Ilmiy maqola uchun
- Giperparametr tanlashda

Empirik qoida:

Kohavi (1995) tadqiqoti:
 k=10 → Ko'p holatlarda optimal
 Bias ham, Variance ham muvozanatlangan

Breiman & Spector (1992):
 k=5 va k=10 → Amalda katta farq yo'q
 Hisoblash resursi muhim bo'lsa → k=5

Xulosa: Katta dataset → k=5, Kichik dataset → k=10

5. Kichik Datasetlarda CV

Kichik dataset ($n < 500$) da CV ayniqsa muhim:

$n=100$, oddiy 80/20 split:

Train=80, Test=20

→ 20 ta namuna bilan baholash → ISHONCHSIZ

→ Bitta "qiyin" namuna skopni 5% tushirishi mumkin

$n=100$, $k=10$ CV:

Har fold: Train=90, Test=10

→ 10 ta fold → Barcha namunalar baholashga kiradi

→ O'rtacha skor ancha ishonchli

$n=100$, $k=5$ CV:

Har fold: Train=80, Test=20

→ 5 ta fold → Barcha namunalar test bo'ladi

→ Yaxshi muvozanat

Leave-One-Out CV (LOOCV) — Eng kichik dataset uchun:

$k = n$ → Har safar 1 ta namuna test

$n=50$:

Iteratsiya 1: Train[2..50], Test[1] → Score₁

Iteratsiya 2: Train[1,3..50], Test[2] → Score₂

...

Iteratsiya 50: Train[1..49], Test[50] → Score₅₀

Final = O'rtacha(Score₁...Score₅₀)

Afzalligi:

→ Har namuna test bo'ladi → Bias minimal

→ Kichik dataset uchun ideal ($n < 50$)

Kamchiligi:

→ n ta model o'rgatiladi → Juda sekin!

→ Yuqori Variance (har model 1 ta namunada baholanadi)

→ n katta bo'lsa → Amaliy emas

6. Stratified K-Fold — Nomutanosib Ma'lumot

Oddiy K-Fold da sinflar nomutanosib bo'lishi mumkin:

Muammo:

100 ta namuna: 90 Sog', 10 Kasal

Tasodifiy fold:

Fold 1: [18 Sog', 2 Kasal] ← To'g'ri

Fold 2: [20 Sog', 0 Kasal] ← Kasal yo'q! ❌

Fold 3: [16 Sog', 4 Kasal] ← Noto'g'ri nisbat

...

Fold 2 da model hech qachon Kasalni ko'rmagan holda test!

→ Baholash noto'g'ri

Stratified K-Fold:

Har foldda sinf nisbati SAQLANADI!

```
Fold 1: [18 Sog', 2 Kasal]    ← 90/10  ✓
Fold 2: [18 Sog', 2 Kasal]    ← 90/10  ✓
Fold 3: [18 Sog', 2 Kasal]    ← 90/10  ✓
...
```

Qachon Stratified:

- Imbalanced ma'lumot (bir sinf kam)
- Binary klassifikatsiya
- Sklearn: StratifiedKFold, cross_val_score default

7. Boshqa CV Turlari

ShuffleSplit:

- Har iteratsiyada tasodifiy yangi bo'linish
- n_splits va test_size mustaqil tanlanadi
- Iteratsiya soni k ga bog'liq emas
- Katta dataset, tez baholash uchun

TimeSeriesSplit:

- Vaqt qatori ma'lumot uchun
- Kelajak ma'lumot hech qachon train da bo'lmaydi!

```
Yil: [2020][2021][2022][2023][2024]
Iter 1: Train[2020]      Test[2021]
Iter 2: Train[2020,2021] Test[2022]
Iter 3: Train[2020..2022]Test[2023]
...
```

Oddiy K-Fold vaqt qatorida XATO:

Train da 2024, Test da 2020 → "Kelajakni ko'rgan" model!

GroupKFold:

- Bir guruh namunalar faqat train YOKI test da
- Misol: Bir bemorning bir necha o'lchovlari
Bir bemorning o'lchovlari split bo'linmasin!

RepeatedKFold:

- K-Fold ni bir necha marta takrorlash
- Har takrorlashda boshqa tasodifiy tartib
- Natija yanada barqaror

8. CV Natijasini To'g'ri Talqin Etish

```
from sklearn.model_selection import cross_val_score, KFold
from sklearn.ensemble import RandomForestClassifier
import numpy as np
```

```
model = RandomForestClassifier(n_estimators=100, random_state=42)
kf = KFold(n_splits=10, shuffle=True, random_state=42)
```

```

skorlar = cross_val_score(
    model, X, y,
    cv=kf,
    scoring="f1"
)

print(f"Har fold:      {skorlar.round(3)}")
print(f"O'rtacha:      {skorlar.mean():.4f}")
print(f"Std og'ish:     {skorlar.std():.4f}")
print(f"95% CI:        {skorlar.mean():.3f} ± {1.96*skorlar.std():.3f}")

```

Natijani talqin etish:
O'rtacha: 0.87 → Model o'rtacha 87% F1 score beradi
Std: 0.02 → Barqaror (±2%)
Std: 0.15 → Beqaror (±15%) → Muammo bor!

Standart og'ish talqini:

Std = 0.02:
 Model turli ma'lumot bo'laklarida bir xil ishlaydi
 → Ishonchli model 

Std = 0.15:
 Ayrim foldda yaxshi, boshqasida yomon
 → Model beqaror 
 → Sabab: Ma'lumot kichik, overfitting, class imbalance

Ikki model taqqoslash:
 Model A: 0.87 ± 0.02
 Model B: 0.89 ± 0.08

Model B o'rtacha yuqori, lekin beqaror
 Model A ishonchli → Amalda A afzal bo'lishi mumkin!

9. Xulosa

K-Fold CV vs Train-Test:
 Train-Test → Bitta bo'linish → Tasodifga bog'liq
 K-Fold → k ta baholash → O'rtacha + Std → Ishonchli

k tanlash:
 k=5 → Tez, katta dataset, standart
 k=10 → Aniq, o'rta dataset, ilmiy tadqiqot
 LOOCV → Juda kichik dataset (n<50), sekin

Kichik dataset:
 → k=10 yoki LOOCV
 → Stratified (sinf nisbatini saqlash)
 → RepeatedKFold (yanada barqaror natija)

CV turlari:
 KFold → Umumiy holat
 StratifiedKFold → Imbalanced, klassifikatsiya
 TimeSeriesSplit → Vaqt qatori

GroupKFold → Guruhli ma'lumot
LOOCV → $n=k$, juda kichik dataset

Natijani talqin:

O'rtacha → Model sifati

Std kichik → Barqaror model ✓

Std katta → Beqaror model ⚠

💡 **Eslab qol:** "Bir martalik train-test — tanga tashlash kabidir. K-fold CV — o'sha tangani 10 marta tashlab, o'rtacha natijani olish kabidir." Standart og'ish kichik bo'lsa — model haqiqatan ishonchli. Faqat o'rtachaga qarab model tanlash — eng ko'p uchraydigan xatolardan biri!

● SAVOL 34:

FEATURE SELECTION (XUSUSIYAT TANLASH) NIMA UCHUN MUHIM? KO'P XUSUSIYAT MODELNI QANDAY ZAIFLASHTIRADI? CORRELATION MATRIX VA FEATURE IMPORTANCE ASOSIDA XUSUSIYAT TANLASH.

1. Feature Selection — Nima Va Nima Uchun?

Tasavvur qiling:

Uy narxini bashorat qilish uchun:

Foydali:

- Maydon (m^2)
- Xonalar soni
- Joylashuv
- Qavat
- Yoshi

Befoyda:

- Egasining ismi
- Ro'yxatga olingan sana
- Egasining tug'ilgan yili
- Qo'shni uyning rangi
- O'tgan yilgi ob-havo

Befoyda xususiyatlar qo'shilsa → Model yomonlashadi!

Nima uchun? → Quyida tushuntiriladi.

Feature Selection ta'rifi:

Barcha mavjud xususiyatlar (p ta)

↓

Feature Selection

↓

Eng muhim xususiyatlar (m ta, $m \ll p$)

Maqsad:

- Model aniqligini oshirish
 - Overfitting kamaytirilishi
 - Tezlikni oshirish
 - Talqin qilishni osonlashtirish
 - Xotirani tejash
-

2. Ko'p Xususiyat Modelni Qanday Zaiflashtiradi?

Muammo 1 — O'lchamlar La'nati (Curse of Dimensionality):

Nazariy asos:

- 1 o'lchovda: 10 ta nuqta → Yaxshi qamrov
- 2 o'lchovda: $10^2 = 100$ ta nuqta kerak
- 10 o'lchovda: 10^{10} ta nuqta kerak!

O'lcham oshgan sari: Ma'lumot "siyraklashadi"

- Nuqtalar bir-biridan uzoqlashadi
- "Yaqin qo'shni" tushunchasi ma'nosiz bo'ladi
- KNN, SVM kabi masofaga asoslangan algoritmlar zaiflanadi

Vizual:

- 1D: ●●●●●●●● (zich)
- 2D: ● ● ● (siyrak)
- 3D: Yanada siyrak...
- 10D: Deyarli bo'sh fazo!

Muammo 2 — Overfitting:

p xususiyat, n kuzatuv:

- $p \ll n$ → Model yaxshi umumlashtiradi
- $p \approx n$ → Model yodlashga moyil
- $p > n$ → Model ALBATTA overfitting qiladi!
(Chiziqli model: Cheksiz ko'p yechim)

Misol:

- $n=100$ kuzatuv, $p=95$ xususiyat
- Chiziqli regressiya: Train $R^2=1.0$, Test $R^2=-0.5$
- Model ma'lumotni yodlagan!

Muammo 3 — Multikolinearlik:

Ikki xususiyat kuchli korrelyatsiyada:

- X_1 = Uy maydoni (m^2)
- X_2 = Xonalar soni (m^2 bilan 0.95 korrelyatsiya)

Model: $Y = \beta_1 X_1 + \beta_2 X_2$

Muammo: X_1 oshsa X_2 ham oshadi → Beta lar beqaror

$\beta_1=+500$, $\beta_2=-200$ yoki $\beta_1=-300$, $\beta_2=+400$

Ikkalasi ham "bir xil" model → Talqin qilib bo'lmaydi!

OLS matritsasi ($X^T X$) → Singular yoki deyarli singular

→ Teskari qilib bo'lmaydi → Koeffitsiyentlar ishonchsiz

Muammo 4 — Shovqin Xususiyatlar:

Tasodifiy xususiyat:

$X_{\text{shovqin}} = np.random.randn(n)$ ← Y bilan hech bog'liqlik yo'q

Lekin: Kichik datasetda TASODIFAN Y bilan korrelyatsiya bo'lishi mumkin
Model bu tasodifiy korrelyatsiyani "haqiqiy naqsh" deb o'rganadi
→ Train yaxshi, Test yomon → Overfitting!

$p=100$ tasodifiy xususiyat:

→ Statistik kutish: ~5 tasi 95% CI da "muhim" ko'rinadi
→ Bular aslida tasodifiy munosabatlar!

Muammo 5 — Hisoblash va Xotira:

p xususiyat, n kuzatuv:

Xotira: $O(n \times p)$

Hisoblash: $O(n \times p^2)$ yoki $O(n \times p \times \log p)$

$p=1,000$: Tez

$p=10,000$: Sekin

$p=100,000$: Juda sekin

$p=1,000,000$: Amaliy emas!

Feature selection → p ni kamaytirish → Tizim samarali

3. Feature Selection Turlari — Umumiy Ko'rinish

Feature Selection usullari:

1. Filter Methods (Filtr):

- Modeldan mustaqil
- Statistik ko'rsatkich asosida
- Tez, lekin model bilan o'zaro ta'sir hisoblanmaydi
- Correlation, Chi-square, ANOVA, Mutual Information

2. Wrapper Methods (O'rab oluvchi):

- Model ishtirokida xususiyatlar tanlanadi
- Aniq, lekin hisoblash ko'p
- RFE, Forward/Backward Selection

3. Embedded Methods (Ichki):

- Model o'zi xususiyat tanlaydi
- O'rgatish jarayonida amalga oshadi
- Lasso, Random Forest Importance, Decision Tree

4. Correlation Matrix — Filtr Usuli

Nazariy asos: Korrelyatsiya koeffitsienti ikki o'zgaruvchi orasidagi chiziqli bog'liqlikni o'lchaydi.

Pearson korrelyatsiyasi:

$$r = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{[\sum(x_i - \bar{x})^2 \cdot \sum(y_i - \bar{y})^2]}}$$

$r \in [-1, +1]$

$r = +1.0 \rightarrow$ Mukammal musbat
 $r = +0.7 \rightarrow$ Kuchli musbat
 $r = +0.3 \rightarrow$ Zaif musbat
 $r = 0.0 \rightarrow$ Bog'liqlik yo'q
 $r = -0.3 \rightarrow$ Zaif manfiy
 $r = -0.7 \rightarrow$ Kuchli manfiy
 $r = -1.0 \rightarrow$ Mukammal manfiy

Correlation Matrix ikki maqsadda ishlatiladi:

Maqsad 1 — Y bilan korrelyatsiya (Feature-Target):

Qaysi xususiyatlar Y ga ko'proq ta'sir qiladi?

Xususiyat	Y bilan r	Muhimmi?
Maydon	+0.85	✓ Ha
Joylashuv	+0.72	✓ Ha
Yoshi	-0.68	✓ Ha
Xonalar soni	+0.61	✓ Ha
Rang	+0.04	✗ Yo'q
Egasi ismi	+0.02	✗ Yo'q

Qoida: $|r| < 0.1 \rightarrow$ Xususiyat deyarli muhim emas
 $|r| > 0.3 \rightarrow$ E'tibor qaratish kerak
 $|r| > 0.5 \rightarrow$ Kuchli bog'liqlik

Maqsad 2 — Xususiyatlar orasida korrelyatsiya (Feature-Feature):

Multikolinearlik aniqlash:

Xususiyat A	Xususiyat B	r	Nima qilish?
Maydon	Xonalar	0.95	Bittasini olib tashlash!
Yoshi	Eskilik	0.88	Bittasini olib tashlash!
Joylashuv	Narx/m ²	0.45	Ikkalasini saqlash mumkin

Qoida: $|r| > 0.8-0.9 \rightarrow$ Bitta xususiyat olib tashlansin

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
```

```
# Korrelyatsiya matritsasi:
corr_matrix = df.corr()
```

```
# 1. Y bilan korrelyatsiya:
y_corr = corr_matrix["Narx"].abs().sort_values(ascending=False)
print(y_corr)
```

```
# 2. Yuqori o'zaro korrelyatsiya:
yuqori_corr = set()
for i in range(len(corr_matrix)):
    for j in range(i):
        if abs(corr_matrix.iloc[i, j]) > 0.85:
            yuqori_corr.add(corr_matrix.columns[i])

print(f"O'chirish uchun: {yuqori_corr}")

# 3. Heatmap:
plt.figure(figsize=(12, 8))
sns.heatmap(
    corr_matrix,
    annot=True, fmt=".2f",
    cmap="coolwarm",
    center=0,
    vmin=-1, vmax=1
)
plt.title("Korrelyatsiya Matritsasi")
plt.show()
```

Correlation ning cheklovi:

Pearson faqat CHIZIQLI bog'liqlikni o'lchaydi!

```
X = [-3,-2,-1,0,1,2,3]
Y = X² = [9,4,1,0,1,4,9]
```

$r(X, Y) \approx 0.0 \leftarrow$ Korrelyatsiya yo'q deydi
 Lekin $Y = X^2 \rightarrow$ Kuchli bog'liqlik bor!

Yechim $\rightarrow$ Spearman korrelyatsiyasi (tartib asosida):
 $\rightarrow$ Nochiziqli monoton bog'liqlikni ham ushlaydi

Yoki $\rightarrow$ Mutual Information:
 $\rightarrow$ Istalgan bog'liqlik turini o'lchaydi

5. Feature Importance — Embedded Usul

Random Forest Feature Importance — eng ishonchli va keng tarqalgan usul.

Nazariy asos (MDI — Mean Decrease Impurity):
 Har bo'lishda Gini kamayishi hisoblanadi
 Qaysi xususiyat bo'lishda ishlatilgan $\rightarrow$ Unga hissa
 Barcha daraxtlar bo'yicha o'rtacha $\rightarrow$ Feature Importance

$$FI(x_j) = \sum_{Tree} \sum_{t \in T(x_j)} [n_t/n \cdot \Delta Gini(t)]$$

Normalizatsiya: $\sum FI = 1.0$

```
from sklearn.ensemble import RandomForestClassifier
import pandas as pd
import matplotlib.pyplot as plt
```

```

model = RandomForestClassifier(n_estimators=200, random_state=42)
model.fit(X_train, y_train)

# Feature importance:
importance = pd.Series(
    model.feature_importances_,
    index=X.columns
).sort_values(ascending=False)

print(importance)
# Maydon      0.342
# Joylashuv   0.218
# Yoshi       0.187
# Xonalar     0.134
# Qavat       0.089
# Rang        0.021
# Egasi_ismi  0.009

# Vizualizatsiya:
importance.plot(kind="barh", figsize=(10, 6), color="steelblue")
plt.xlabel("Muhimlik darajasi")
plt.title("Random Forest Feature Importance")
plt.tight_layout()
plt.show()

# Muhim xususiyatlarni tanlash:
muhim_xususiyatlar = importance[importance > 0.05].index
X_tanlab_olingan = X[muhim_xususiyatlar]

```

Feature Importance ning muammosi — Bias:

MDI muammosi:

Yuqori kardinallik (ko'p noyob qiymat) bo'lgan xususiyatlar haddan oshiq baholanadi!

Misol: "ID" ustuni → 1000 ta noyob qiymat
 → Ko'p bo'lish imkoni → Yuqori FI
 → Lekin aslida befoyda!

Yechim — Permutation Importance:

1. Asl model skori: $S_0 = 0.89$
2. Xususiyat j ni aralashtir
3. Yangi skor: $S_j = 0.71$
4. $FI(j) = S_0 - S_j = 0.18 \leftarrow$ Muhim!

5. Boshqa xususiyatni aralashtir
6. Yangi skor: $S_k = 0.88$
7. $FI(k) = S_0 - S_k = 0.01 \leftarrow$ Muhim emas!

```
from sklearn.inspection import permutation_importance
```

```

perm_imp = permutation_importance(
    model, X_test, y_test,
    n_repeats=30,          # 30 marta aralashtirish → Barqaror natija
    random_state=42,
    scoring="f1"
)

```

)

```
perm_df = pd.DataFrame({
    "xususiyat": X.columns,
    "muhimlik": perm_imp.importances_mean,
    "std": perm_imp.importances_std
}).sort_values("muhimlik", ascending=False)

print(perm_df)
```

6. RFE — Recursive Feature Elimination

Wrapper usuli — model ishtirokida xususiyatlar ketma-ket olib tashlanadi:

Jarayon:

1. Barcha p xususiyat bilan model o'rgatish
2. Eng muhim emas xususiyatni olib tashlash
3. Qolgan xususiyatlar bilan qayta o'rgatish
4. Kerakli soni qolguncha takrorlash

$p=10$ dan $m=5$ ga:

Iteratsiya 1: [1,2,3,4,5,6,7,8,9,10] → X_7 olib tashlash
Iteratsiya 2: [1,2,3,4,5,6,8,9,10] → X_2 olib tashlash
Iteratsiya 3: [1,3,4,5,6,8,9,10] → X_9 olib tashlash
Iteratsiya 4: [1,3,4,5,6,8,10] → X_1 olib tashlash
Iteratsiya 5: [3,4,5,6,8,10] → 5 ta qoldi. STOP!

Natija: {3,4,5,6,8,10} → Eng muhim 6 ta xususiyat

```
from sklearn.feature_selection import RFE, RFECV
from sklearn.linear_model import LogisticRegression
```

RFE – Nechta xususiyat tanlanishini belgilash:

```
rfe = RFE(
    estimator=LogisticRegression(),
    n_features_to_select=5,
    step=1 # Har iteratsiyada 1 ta olib tashlash
)
rfe.fit(X_train, y_train)
```

```
print(f"Tanlangan: {X.columns[rfe.support_].tolist()}")
```

```
print(f"Reyting: {rfe.ranking_}")
```

Reyting=1 → Tanlangan, Reyting=2 → Birinchi olib tashlangan...

RFECV – Optimal sonni avtomatik topish:

```
rfevcv = RFECV(
    estimator=LogisticRegression(),
    cv=5,
    scoring="accuracy"
)
rfevcv.fit(X_train, y_train)
print(f"Optimal xususiyatlar soni: {rfevcv.n_features_}")
```

7. Lasso — Avtomatik Feature Selection

L1 regularizatsiya (Lasso) — ba'zi koeffitsiyentlarni aynan 0 ga tushiradi → Avtomatik feature selection!

Lasso maqsad funksiyasi:

$$\text{Min: } \text{MSE} + \lambda \cdot \sum |\beta_j|$$

λ kichik → Kam koeffitsiyent 0 → Ko'p xususiyat saqlanadi

λ katta → Ko'p koeffitsiyent 0 → Kam xususiyat saqlanadi

Nima uchun L1 "0 ga tushiradi", L2 emas?

L1 (Lasso): "Olmoc" shaklidagi cheklov → Burchaklarda $\beta=0$

L2 (Ridge): "Doira" shaklidagi cheklov → Hech qachon aynan 0 emas

Geometrik: MSE konturlari "olmos" burchagiga tegadi →
Bir β aynan 0 bo'ladi!

```
from sklearn.linear_model import LassoCV
import numpy as np
```

```
# LassoCV — optimal  $\lambda$  ni cross-validation bilan topadi:
```

```
lasso = LassoCV(cv=5, random_state=42)
lasso.fit(X_train, y_train)
```

```
# Nol bo'lmagan koeffitsiyentlar → Tanlangan xususiyatlar:
```

```
tanlangan = X.columns[lasso.coef_ != 0]
olib_tashlangan = X.columns[lasso.coef_ == 0]
```

```
print(f"Tanlangan ({len(tanlangan)} ta): {tanlangan.tolist()}")
print(f"Olib tashlangan ({len(olib_tashlangan)} ta): {olib_tashlangan.tolist()}")
print(f"Optimal lambda: {lasso.alpha_:.4f}")
```

8. SelectKBest — Statistik Filtr

```
from sklearn.feature_selection import SelectKBest, f_classif, mutual_info_classif
```

```
# f_classif → ANOVA F-statistika (chiziqli bog'liqlik)
```

```
# mutual_info_classif → Mutual Information (har qanday bog'liqlik)
```

```
selector = SelectKBest(
    score_func=mutual_info_classif,
    k=10          # Eng yaxshi 10 ta
)
selector.fit(X_train, y_train)
```

```
# Skorlar:
```

```
scores = pd.Series(
    selector.scores_,
    index=X.columns
).sort_values(ascending=False)
```

```
X_yangi = selector.transform(X_train)
print(f"Yangi o'lcham: {X_yangi.shape}")
```

9. Amaliy Strategiya — Qaysi Usulni Tanlash?

Dataset holati	→ Tavsiya
Tez prototip	→ Correlation + RF Importance
Ko'p kategoriyali	→ Mutual Information
Chiziqli model	→ Lasso (avtomatik)
Kichik dataset	→ RFE + CV (aniq, lekin sekin)
Katta dataset	→ RF Importance (tez)
Vaqt qatori	→ Lag korrelyatsiyasi
Matn ma'lumot	→ TF-IDF + Chi-square

Umumiy qoida:

1. Korrelyatsiya → Tez filtr (ochiq yomon xususiyatlar)
2. RF Importance → O'rtacha filtr
3. Permutation → Aniq baholash
4. RFE/Lasso → Final tanlash

10. Xulosa

Nima uchun muhim:

- Ko'p xususiyat → O'lchamlar la'nati
- Ko'p xususiyat → Overfitting ($p > n$)
- Ko'p xususiyat → Multikolinearlik
- Ko'p xususiyat → Shovqin xususiyatlar
- Ko'p xususiyat → Sekin, ko'p xotira

Usullar:

Filter:

- Korrelyatsiya → Y bilan $r > 0.1$ saqlash
X-X orasida $r < 0.85$ saqlash
- SelectKBest → Statistik test asosida

Embedded:

- RF Importance → MDI (tez, lekin kardinallikka sezgir)
- Permutation → Ishonchli, sekin
- Lasso (L1) → Koeffitsiyentlarni 0 ga tushiradi

Wrapper:

- RFE → Ketma-ket olib tashlash
- RFECV → Optimal son avtomatik

Asosiy qoidalar:

- ✓ `fit()` faqat train da → Data Leakage yo'q
- ✓ Pipeline ichida → Xavfsiz CV
- ✓ Bir usul yetarli emas → Kombinatsiya ishlatish
- ✓ Domain bilimi → Eng yaxshi feature selection

💡 **Eslab qol:** Feature Selection — "Axlal kirs, axlat chiqadi" tamoyiliga qarshi kurash.
1000 ta xususiyatdan 20 ta to'g'ri tanlangan xususiyat, 1000 ta bilan o'rgatilgan modeldan

ko'ra **ancha yaxshi** natija berishi mumkin. Ko'p ma'lumot emas — **to'g'ri ma'lumot** muhim!

● SAVOL 35:

SKLEARN PIPELINE NIMA VA NIMA UCHUN ISHLATILADI? PREPROCESSING VA MODEL O'RGATISHNI BITTA OBYEKTGA BIRLASHTIRISH AFZALLIKLARI VA DATA LEAKAGE NI OLDINI OLISH.

1. Pipeline — Asosiy G'oya

Mashinali o'rganish jarayoni odatda ko'p qadamdan iborat:

```
Xom ma'lumot
↓
Bo'sh qiymatlarni to'ldirish (Imputation)
↓
Miqyoslash (Scaling)
↓
Kodlash (Encoding)
↓
O'lcham kamaytirish (PCA)
↓
Model o'rgatish
↓
Bashorat
```

Pipeline — Bu BARCHA qadamlarni BITTA ob'ektga bog'lash!

```
pipe.fit(X_train, y_train)    ← Barcha qadamlar ketma-ket
pipe.predict(X_test)         ← Barcha qadamlar avtomatik
```

2. Data Leakage — Eng Muhim Tushuncha

Data Leakage — test ma'lumoti haqidagi axborot train jarayoniga "sizib o'tishi". Bu modelni yolg'on ravishda yaxshi ko'rsatadi.

Analogiya:

- Talaba imtixon savollarini oldindan ko'radi
- Imtihonda 100% oladi
- Lekin haqiqiy bilimi yo'q!

Modelda:

- Test ma'lumotining statistikasi train jarayonida ishlatiladi
- Model "kelajakni ko'rgan"
- Train/Test skori yuqori, lekin haqiqiy bashorat yomon

Data Leakage ning eng keng tarqalgan sababi:

❌ KLASSIK DATA LEAKAGE:

```
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
```

```
scaler = StandardScaler()
```

XATO: Butun datasetda fit!

```
X_scaled = scaler.fit_transform(X) # ← Test ma'lumoti ham "ko'rildi"!
```

Cross-validation:

```
skorlar = cross_val_score(
    LogisticRegression(), X_scaled, y, cv=5
)
```

→ Har fold da test ma'lumoti allaqachon scale qilingan

→ Scaler test ma'lumotining μ va σ ni "biladi"

→ Natija: Optimistik, haqiqiy emas!

Nima uchun bu leakage:

5-fold CV, to'g'ri:

```
Fold 1: Train=[2,3,4,5] → scaler.fit(X[2,3,4,5])
                                →  $\mu=23.1$ ,  $\sigma=4.5$  (faqat train dan)
    Test=[1] → scaler.transform(X[1])
                                (NOMA'LUM statistika bilan scale)
```

5-fold CV, noto'g'ri (leakage):

```
Avval: scaler.fit_transform(X_BARCHASI)
        →  $\mu=23.5$ ,  $\sigma=4.8$  (test MA'LUMOTI ham ichida!)
Fold 1: Test=[1] → allaqachon scale qilingan
        (Scaler test nuqtasining qiymatini "bilgan")
```

3. Pipeline — Data Leakage ni Qanday Oldini Oladi?

Pipeline + Cross-Validation:

```
pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression())
])
```

```
cross_val_score(pipe, X, y, cv=5)
```

Fold 1 ichida nima bo'ladi:

```
Train=[2,3,4,5]:
    pipe.fit(X[2,3,4,5], y[2,3,4,5])
    → scaler.fit_transform(X[2,3,4,5]) ← Faqat train!
    → model.fit(scaled_X_train)
```

```
Test=[1]:
    pipe.predict(X[1])
    → scaler.transform(X[1]) ← fit() EMAS! transform()
    → model.predict(scaled_X_test)
```

Har fold uchun scaler QAYTADAN fit qilinadi!
Test ma'lumoti hech qachon fit() ga kirmaydi! 

4. Pipeline Tuzilishi

```
from sklearn.pipeline import Pipeline
```

```
pipe = Pipeline([
    ("qadamNom1", transformer1),
    ("qadamNom2", transformer2),
    ("modelNom", estimator)
])
```

Qoidalar:

- # 1. Ro'yxatdagi OXIRGI element → Estimator (fit+predict)*
- # 2. Boshqa barcha elementlar → Transformer (fit+transform)*
- # 3. Nomlar – ixtiyoriy, lekin unikal va ma'noli bo'lsin*
- # 4. Har element: fit() va transform() metodiga ega bo'lishi kerak*
- # (Oxirgi element: fit() va predict())*

Pipeline ichki ishlash mexanizmi:

```
pipe.fit(X_train, y_train):
    1. transformer1.fit_transform(X_train)          →  $X_1$ 
    2. transformer2.fit_transform( $X_1$ )              →  $X_2$ 
    3. estimator.fit( $X_2$ , y_train)

pipe.predict(X_test):
    1. transformer1.transform( $X_{test}$ )              →  $X_1$ 
       (fit() emas! train statistikasi ishlatiladi)
    2. transformer2.transform( $X_1$ )                  →  $X_2$ 
    3. estimator.predict( $X_2$ )                       →  $\hat{y}$ 

pipe.score(X_test, y_test):
    1-2. Yuqoridagi transform qadamlar
    3. estimator.score( $X_2$ , y_test)                → Skor
```

5. make_pipeline — Qisqartma Usul

```
from sklearn.pipeline import make_pipeline
```

Uzun usul:

```
pipe = Pipeline([
    ("standardscaler", StandardScaler()),
    ("polynomialfeatures", PolynomialFeatures(degree=2)),
    ("linearregression", LinearRegression())
])
```

Qisqa usul (nomlar avtomatik):

```
pipe = make_pipeline(
    StandardScaler(),
    PolynomialFeatures(degree=2),
    LinearRegression())
```

```
)  
# Nomlar: "standardscaler", "polynomialfeatures", "Linearregression"  
# (Klass nomi kichik harflarda)
```

6. Pipeline Giperparametrlarga Kirish

```
pipe = Pipeline([  
    ("scaler", StandardScaler()),  
    ("poly", PolynomialFeatures()),  
    ("model", Ridge())  
)  
  
# Giperparametrlarga kirish: "qadamNom__parametrNom"  
pipe.set_params(poly__degree=3)  
pipe.set_params(model__alpha=0.1)  
  
# get_params:  
print(pipe.get_params())  
# {  
#   'scaler': StandardScaler(),  
#   'scaler__copy': True,  
#   'poly': PolynomialFeatures(),  
#   'poly__degree': 3,  
#   'model': Ridge(),  
#   'model__alpha': 0.1,  
#   ...  
# }  
  
# GridSearchCV bilan:  
from sklearn.model_selection import GridSearchCV  
  
params = {  
    "poly__degree": [1, 2, 3],  
    "model__alpha": [0.01, 0.1, 1.0, 10.0],  
}  
  
grid = GridSearchCV(pipe, params, cv=5, scoring="r2")  
grid.fit(X_train, y_train)  
  
print(f"Eng yaxshi: {grid.best_params_}")  
# {'poly__degree': 2, 'model__alpha': 0.1}
```

7. Murakkab Pipeline — ColumnTransformer bilan

Haqiqiy ma'lumotda har ustun turi boshqacha ishlov talab qiladi:

```
from sklearn.compose import ColumnTransformer  
from sklearn.preprocessing import StandardScaler, OneHotEncoder  
from sklearn.impute import SimpleImputer  
from sklearn.ensemble import RandomForestClassifier  
from sklearn.pipeline import Pipeline  
  
# Ustunlar bo'yicha ajratish:
```

```

son_ustunlar = ["yosh", "daromad", "tajriba_yil"]
kat_ustunlar = ["shahar", "kasb", "ta'lim"]

# Raqamli ustunlar uchun:
son_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

# Kategoriyali ustunlar uchun:
kat_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(
        handle_unknown="ignore",
        sparse_output=False
    ))
])

# ColumnTransformer:
preprocessor = ColumnTransformer([
    ("son", son_pipe, son_ustunlar),
    ("kat", kat_pipe, kat_ustunlar)
], remainder="drop") # Qolgan ustunlarni tashla

# To'liq Pipeline:
to_liq_pipe = Pipeline([
    ("prep", preprocessor),
    ("model", RandomForestClassifier(
        n_estimators=100,
        random_state=42
    ))
])

# Ishlatish – oddiy model kabi:
to_liq_pipe.fit(X_train, y_train)
y_pred = to_liq_pipe.predict(X_test)
skor = to_liq_pipe.score(X_test, y_test)

print(f"Accuracy: {skor:.4f}")

```

8. Pipeline Afzalliklarining To'liq Ro'yxati

Afzallik 1 — Data Leakage himoyasi:

Yuqorida batafsil tushuntirildi.
Eng muhim afzallik – noto'g'ri baholashdan himoya.

Afzallik 2 — Kod sodaligi:

```

# Pipeline siz: 15+ qadam, takror kod
scaler.fit(X_train)
X_train_s = scaler.transform(X_train)
pca.fit(X_train_s)
X_train_p = pca.transform(X_train_s)

```

```

model.fit(X_train_p, y_train)

X_test_s = scaler.transform(X_test)  # transform! fit emas
X_test_p = pca.transform(X_test_s)
y_pred = model.predict(X_test_p)

# Pipeline bilan: 3 qadam
pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)

```



Afzallik 3 — Deployment (ishlab chiqarish) uchun:

```

import joblib

# Butun pipeline ni saqlash:
joblib.dump(to_liq_pipe, "model.pkl")

# Boshqa joyda yuklash:
yuklangan = joblib.load("model.pkl")

# Yangi ma'lumot:
yangi_xom = pd.DataFrame({
    "yosh": [35], "daromad": [50000],
    "tajriba_yil": [8], "shahar": ["Toshkent"],
    "kasb": ["Muhandis"], "ta'lim": ["Bakalavr"]
})

# Preprocessing avtomatik!
bashorat = yuklangan.predict(yangi_xom)

```

Afzallik 4 — Algoritm almashtirishda osonlik:

```

# Faqat bitta qatorni o'zgartirish:
pipe = Pipeline([
    ("prep", preprocessor),
    ("model", LogisticRegression())  # Bu qator
])

# Boshqa algoritm sinash:
pipe.set_params(model=RandomForestClassifier())
pipe.set_params(model=GradientBoostingClassifier())
pipe.set_params(model=SVC())
# Preprocessing qadamlar O'ZGARMAYDI!

```

Afzallik 5 — GridSearchCV integratsiyasi:

```

# Preprocessing + Model giperparametrlari BIRGA qidiriladi:
params = {
    "prep__son_scaler_with_std": [True, False],
    "prep__kat_encoder_drop": [None, "first"],
    "model__n_estimators": [50, 100, 200],
    "model__max_depth": [3, 5, None],
}

# Bu Pipeline siz MUMKIN EMAS (Data Leakage bo'lardi)!

```

9. Pipeline Debugging — Oraliq Natijalar

Qadamlar bo'yicha natijani ko'rish:

```
pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("pca", PCA(n_components=5)),
    ("model", LogisticRegression())
])
pipe.fit(X_train, y_train)
```

Oraliq qadam natijasi:

```
X_scaled      = pipe[:-1].transform(X_train)    # Oxirgi qadamsiz
X_after_scal  = pipe[:1].transform(X_train)     # Faqat birinchi qadam
X_after_pca   = pipe[:2].transform(X_train)     # Ikki qadam
```

Qadam ob'ektiga kirish:

```
scaler_obj = pipe.named_steps["scaler"]
pca_obj    = pipe.named_steps["pca"]
model_obj  = pipe.named_steps["model"]
```

```
print(f"PCA tushuntirilgan dispersiya: {pca_obj.explained_variance_ratio_}")
print(f"Model koeffitsiyentlari: {model_obj.coef_}")
```

10. Xulosa

Pipeline = Preprocessing + Model → BITTA ob'ekt

Data Leakage:

- Test statistikasi train ga "sizib o'tishi"
- Pipeline → Har folda transformer qaytadan fit
- Test hech qachon fit() ga kirmaydi ✅

Afzalliklari:

- ✅ Data Leakage himoyasi (eng muhim!)
- ✅ Kod soda va toza
- ✅ Deployment oson (bitta .pkl fayl)
- ✅ Algoritm almashtirishda faqat 1 qator
- ✅ GridSearchCV bilan preprocessing ham optimallasadi

Asosiy qoidalar:

- fit() → FAQAT train da
- transform() → Train va Test da
- Pipeline → DOIM ishlatish
- ColumnTransformer → Har ustun turi uchun

Giperparametrlarga kirish:

"qadamNom__parametrNom" → set_params() va GridSearchCV

💡 **Eslab qol:** Pipeline ishlatmasdan "to'g'ri" cross-validation qilish deyarli mumkin emas. Hatto tajribali data scientistlar ham Pipeline siz kod yozganda data leakage qiladilar. Sklearn loyihachilari bu sababdan Pipeline ni yaratishgan — uni doim ishlatish eng yaxshi amaliyot!